Race Conditions
Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

February 17, 2025



Instructor: Thomas Feys

1 Introduction

This lab focuses on solving the problem of finding a maximum value in an array in three different ways. Using the CPU, GPU with both atomic operations and reduction.

2 Solution

2.1 Array Generation

To ensure consistency in our results across all tests, we initialised an array starting at zero and ending at N-1. This makes it easy to spot differences between solutions.

```
    const int N = 10000;
    vector<int> h_arr(N);

    // Init array
    for (int i = 0; i < N; i++) {
        h_arr[i] = i;
    }
```

2.2 Results

As shown in Figure 1, the three programs find their maximum values in the array (of size 4000).

```
   Creating library max_Reduction.lib and object max_Reduction.exp
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_CPU.exe
Maximum waarde: 3999
Uitvoeringstijd: 0.0175963 seconden
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_Atomic.exe
Maximum waarde: 3999
Uitvoeringstijd: 0.0003611 seconden
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_reduction.exe
Maximum waarde: 3999
Uitvoeringstijd: 0.0004611 seconden
```

Figure 1: Results array size: 4000

When we increase the array size to 10000, we notice little difference in the timing. Except for the CPU, which increases about 6 times.

```
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_CPU.exe
Maximum waarde: 9999
Uitvoeringstijd: 0.111377 seconden
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_Atomic.exe
Maximum waarde: 9999
Uitvoeringstijd: 0.0004426 seconden
PS C:\Users\Thiba\OneDrive - KU Leuven\SchoolProjecten\KU Leuven\Master\Geavanceerde Computer Architectuur\Labos\Labo2> .\max_reduction.exe
Maximum waarde: 9999
Uitvoeringstijd: 0.0004813 seconden
```

Figure 2: Results array size: 10k

2.3 Limited block size

As shown in the previous lab, the block size cannot be greater than 1024. The solution for processing larger array sizes is thus adding multiple blocks.

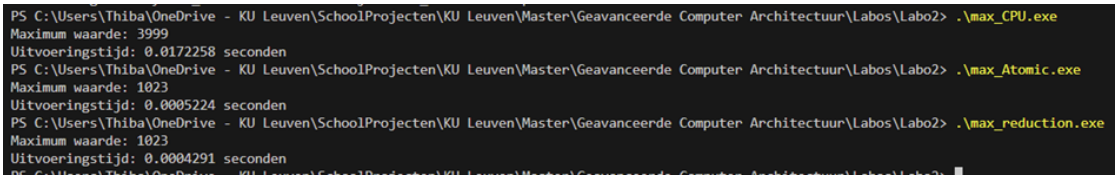


Figure 3: Enter Caption

2.4 Expanding using blocks

As mentioned previously when dealing with an input size greater than 1024 elements. We make use of multiple blocks and avoid using atomic operations. We make use of shared memory for each block. Where the maximum value for each block will be found. When the max value is found the blocks will return this value in the global shared memory, where the CPU will then search for the resulting max value.

2.5 Influence of treads per block

When experimenting with the amount of threads per block we noticed a small difference when using a small amount of threads per block rather than a bigger amount. The extra amount of time it takes to find the result is due to finding the maximum for each block on the CPU (CPU finds the max of all blocks after all blocks have calculated their own max). That's why a large amount of blocks (a small amount of threads per block) will have a bigger delay.

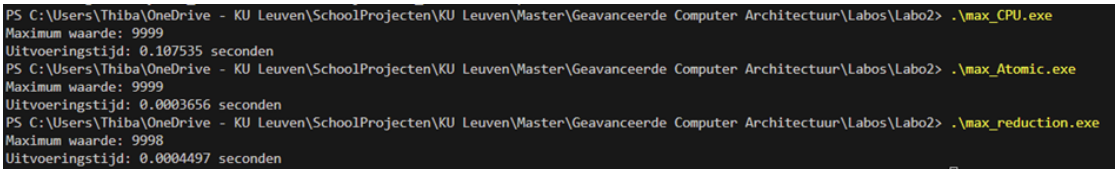


Figure 4: 10 threads per block



Figure 5: 1024 threads per block

2.6 Solve 2D array

We believe there are two different ways a 2D array can be solved:

- Put each row in a block
  If the rows are too long for this, we can add another loop in the kernel so we find the max in each block in the row, find the max of all blocks in the row end then find the max of all rows.
- Transform 2D array to 1D
  By transforming the 2D array to a 1D array, we can just execute the same code as in this lab.

2.7 Code

2.7.1 CPU

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace std::chrono;

int findMaxNestedLoop(const vector<int>& arr) {
    int maxVal = arr[0];
    for (size_t i = 0; i < arr.size(); i++) {
        for (size_t j = 0; j <= i; j++) {
            if (arr[j] > maxVal) {
                maxVal = arr[j];
            }
        }
    }
    return maxVal;
}

int main() {
    const int N = 10000;  // Verander deze waarde voor verschillende tests
    vector<int> arr(N);

    for (int i = 0; i < N; i++) {
        arr[i] = i;
    }

    auto start = steady_clock::now();
    int maxVal = findMaxNestedLoop(arr);
    auto end = steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end-start};

    cout << "Maximum waarde: " << maxVal << endl;
    cout << "Uitvoeringstijd: " << elapsed_seconds.count() << " seconden" << endl;

    return 0;
}
```

2.7.2 GPU - Atomic

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;

__global__ void findMaxAtomic(int* d_arr, int* d_max, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        atomicMax(d_max, d_arr[idx]); // update with atomic operation
    }
}

int main() {
    const int N = 10000;
    vector<int> h_arr(N);

    // Init array
    srand(time(0));
    for (int i = 0; i < N; i++) {
        h_arr[i] = i;
    }

    // memory alloc
    int* d_arr;
    int* d_max;
    int h_max = INT_MIN;

    cudaMalloc(&d_arr, N * sizeof(int));
    cudaMalloc(&d_max, sizeof(int));
    cudaMemcpy(d_arr, h_arr.data(), N * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_max, &h_max, sizeof(int), cudaMemcpyHostToDevice);

    const auto start = chrono::steady_clock::now();

    // Execute kernel
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    findMaxAtomic<<<numBlocks, blockSize>>>(d_arr, d_max, N);
    cudaDeviceSynchronize();

    const auto end = chrono::steady_clock::now();
    const chrono::duration<double> elapsed_seconds{end - start};

    // get result
    cudaMemcpy(&h_max, d_max, sizeof(int), cudaMemcpyDeviceToHost);

    cout << "Maximum waarde: " << h_max << endl;
    cout << "Uitvoeringstijd: " << elapsed_seconds.count() << " seconden" << endl;

    cudaFree(d_arr);
    cudaFree(d_max);

    return 0;
}
```

2.7.3 GPU - Reduction

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;

// Kernel function to find max value in array
__global__ void findMaxReduction(int* d_arr, int* d_max, int N) {
    extern __shared__ int sdata[]; // Shared memory for thread in block
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // load global data into shared memory
    if (idx < N) {
        sdata[tid] = d_arr[idx];
    } else {
        sdata[tid] = INT_MIN;
    }
    __syncthreads();

    // Reduction in shared memory (find max)
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = max(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }

    // Write result for this block to global memory (d_max)
    if (tid == 0) {
        d_max[blockIdx.x] = sdata[0];
    }
}

int main() {
    const int N = 10000;
    vector<int> h_arr(N);

    // init array
    for (int i = 0; i < N; i++) {
        h_arr[i] = i;
    }

    // mem allocation
    int* d_arr;
    int* d_max;
    int h_max;

    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    vector<int> h_max_partial(numBlocks);

    cudaMalloc(&d_arr, N * sizeof(int));
    cudaMalloc(&d_max, numBlocks * sizeof(int));
    cudaMemcpy(d_arr, h_arr.data(), N * sizeof(int), cudaMemcpyHostToDevice);


    const auto start = chrono::steady_clock::now();
```

```
61
62      findMaxReduction<<<numBlocks, blockSize, blockSize * sizeof(int)>>>(d_arr,
        d_max, N);
63      cudaDeviceSynchronize();
64
65      const auto end = chrono::steady_clock::now();
66      const chrono::duration<double> elapsed_seconds{end - start};
67
68      // Get result & finish on CPU
69      cudaMemcpy(h_max_partial.data(), d_max, numBlocks * sizeof(int),
        cudaMemcpyDeviceToHost);
70
71      // CPU-reduction of partial results
72      h_max = h_max_partial[0];
73      for (int i = 1; i < numBlocks; i++) {
74          h_max = max(h_max, h_max_partial[i]);
75      }
76
77      cout << "Maximum waarde: " << h_max << endl;
78      cout << "Uitvoeringstijd: " << elapsed_seconds.count() << " seconden" << endl;
79
80      cudaFree(d_arr);
81      cudaFree(d_max);
82
83      return 0;
84  }
```